3

Robuust ontwerp in Python

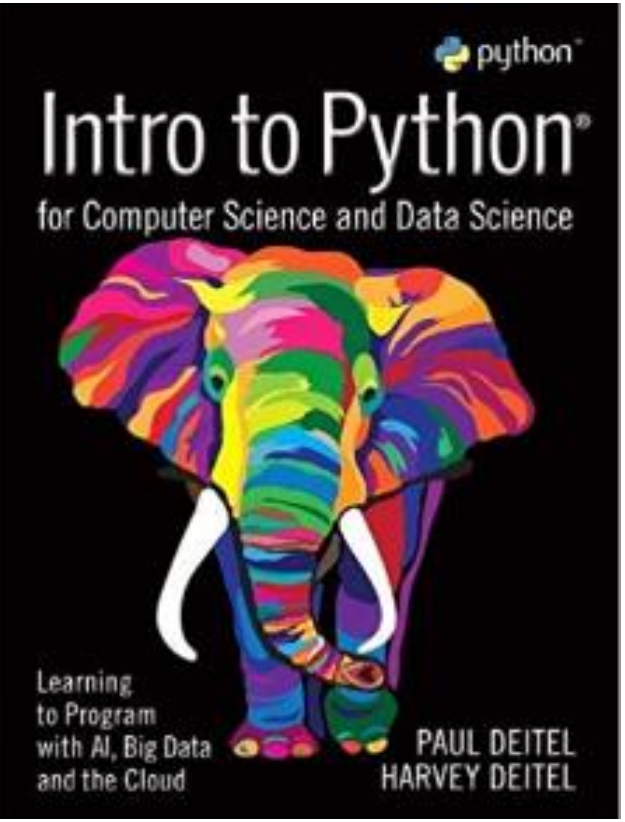
© 2008 Pearson Education, Inc. All rights reserved.

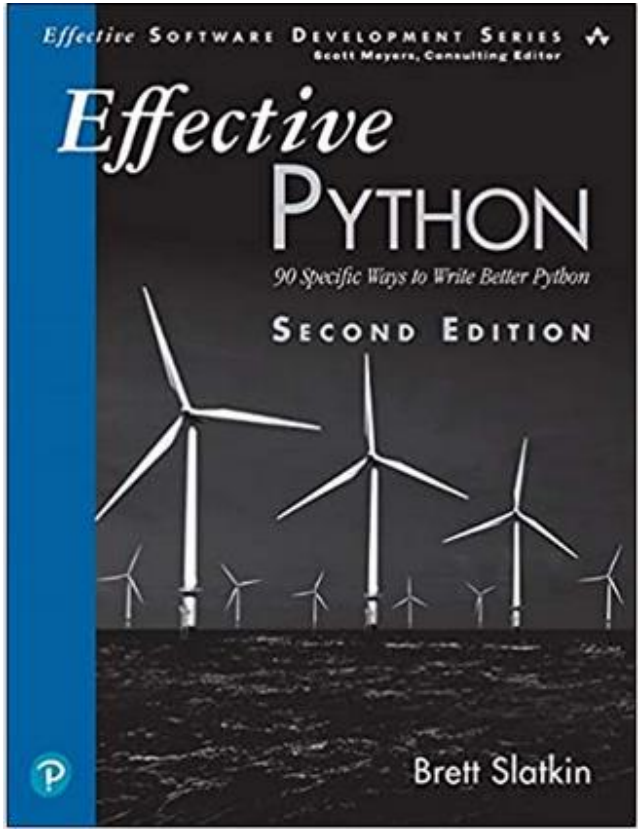
Inhoud

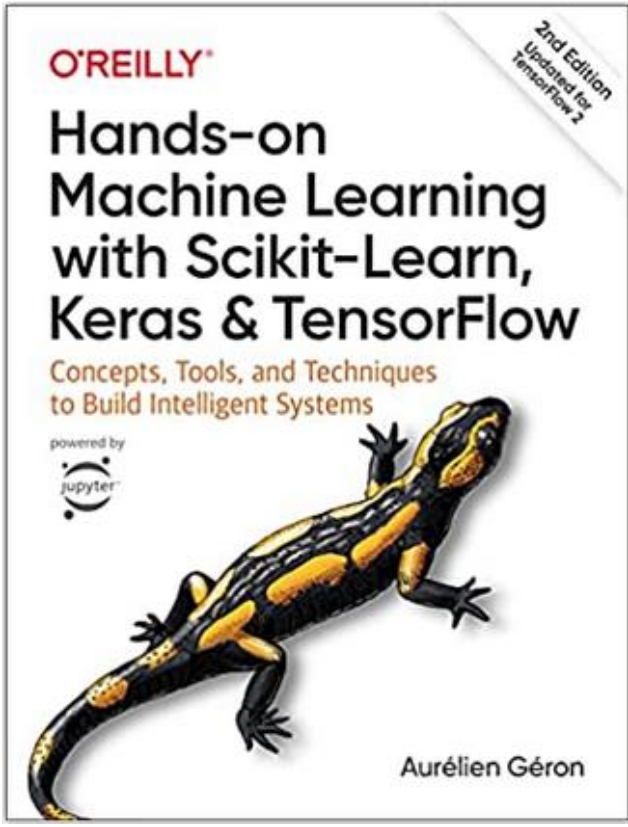
1. Inleiding
2. Programmeren onder Linux
3. Robuust ontwerp in Python
   - 3.1 Pythonic thinking en de PEP8 style
   - 3.2 Functions
   - 3.3 Comprehensions & Generators
   - 3.4 Exception handling
   - 3.5 Polymorfisme
   - 3.6 Performance
   - 3.7 Testing & Debugging

© 2008 Pearson Education, Inc. All rights reserved.

Referenties







2008 Pearson Education, Inc. All rights reserved.

3. Robuust ontwerp

Ontwerp impliceert een **doordachte planning** vooraleer men begint te coderen. Programmeren is 80% planning en testing en 20% coderen.

Hoe vertaalt men probleem uit realiteit in computercode → Software project

Robuust: immuun voor fouten en makkelijk onderhoudbaar en hergebruikbaar

- Fouten: logische (planning), implementatie (bugs), acties van gebruiker
- Hergebruik: modulair, compiler en systeemonafhankelijk

Efficient: gebruik zo goed mogelijk de resources (geheugen, CPU, netwerk, …)

Deze cursus is geen OOP design cursus, maar in deze les geven we een 15-tal design rules mee uit het boek “Effective Python”

© 2008 Pearson Education, Inc. All rights reserved.

3. Strategie

Software ontwerp is duur, kost veel tijd en gebeurt in groepsverband: grote ontwikkeling in standardisatie van methoden en tools:

- Software development methodology: http://en.wikipedia.org/wiki/Software_development_methodology
- Computer aided software engineering: http://en.wikipedia.org/wiki/Computer-aided_software_engineering
- UML: Unified Modeling Language. Grafische methode voor het ontwerpen/analyseren van object oriented systemen
- Design patterns en rules: Optimale oplossingsmodellen voor veel voorkomende situaties
- We behandelen hierna een aantal rules/patterns geordend volgens complexiteit

© 2008 Pearson Education, Inc. All rights reserved.

3. Class design is type design

Merendeel van je Python programmer activiteit is het verbeteren en uitbreiden van beshikbare types. Je moet methods en operators overloaden, initialiseren, algoritmes implementeren, …

Goede types hebben

- Natuurlijke syntax
- Intuitief gebruik
- Efficiente implementaties

Het ontwerp van een nieuwe klasse veronderstelt het beantwoorden van een aantal vragen:

© 2008 Pearson Education, Inc. All rights reserved.

3. Type design

- **Hoe moeten objecten van jouw type worden gecreeerd en vernietigd?** Heeft invloed op je __new__() en __del__() ‘dunder’ methods
- **Hoe moet object initialisatie verschillen van object assignment?** Bepaalt het verschil tussen je __init__() en __copy__() en __deepcopy__()
- **Wat zijn de beperkingen op de toegelaten waarden/toestanden van je nieuwe type?** Bepaalt je error en bounds checking, vooral in constructors, assignment operators en setter methods.
- **Past je nieuwe type in een inheritance hierarchie?** Ge je inheritance, polymorfisme of compositie gebruiken?
- **Welke conversies zijn toegelaten voor je nieuwe type?** Heeft invloed op je __repr__() en __str__() dunders

© 2008 Pearson Education, Inc. All rights reserved.

3. Type design

- **Welke operators en functies zijn zinvol voor je type?** Denk ook na welke precies class methods moeten zijn en welke niet.
- **Wie krijgt toegang tot je class attributes?** Gebruik de naming conventions _name voor niet publiek te gebruiken attributes en __name voor ‘private’ attributes
- **Wat is de verborgen interface van je klasse?** Hoe efficient en robuust wil je ze maken?
- **Hoe algemeen is je nieuwe type?** Definieer je ineens een hele familie van nieuwe types? Wil je duck taping gebruiken?
- **Heb je echt een nieuw type nodig?** Als je enkel wat beperkte extra functionaliteit aan een bestaand type wil toevoegen, kan dat misschien via externe functies.

© 2008 Pearson Education, Inc. All rights reserved.

3.1 Pythonic thinking

- Het is (nog) steeds mogelijk om Python programs's te schrijven in de stijl van een andere programmeertaal (C++, Java, …)
- De Python syntax laat dit in principe toe, maar bevat groot aantal unieke concepten die aan Python een eigen stijl geven: Pythonic style
- Python syntax is gedurende de Jaren sterk uitgebreid met nieuwe concepten: gebruik ten allen tijde Python 3.8 versie of hoger (zeker niet de obsolete Python 2)

```
(mypython) uauser@VMFys:~/code/mypython$ python --version
Python 3.9.5
(mypython) uauser@VMFys:~/code/mypython$ python
Python 3.9.5 (default, Nov 23 2021, 15:27:38)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> print(sys.version_info)
sys.version_info(major=3, minor=9, micro=5, releaselevel='final', serial=0)
>>> print(sys.version)
3.9.5 (default, Nov 23 2021, 15:27:38)
[GCC 9.3.0]
>>>
```

© 2008 Pearson Education, Inc. All rights reserved.

3.1 PEP-8 style guide

- Python Enhanced Proposal #8 (PEP-8) bevat alle details/afspraken over degelijk coderen in Python
- Consistente en gestandardiseerde stijl maakt je code aantrekkelijker voor andere professionals, gemakkelijk leesbaar, bevordert samenwerking, …
- Zie: https://www.python.org/dev/peps/pep-0008/
- Belangrijkste aanbevelingen: zie hierna

© 2008 Pearson Education, Inc. All rights reserved.

3.1 PEP-8 style guide

Whitespace

In Python, whitespace is syntactically significant. Python programmers are especially sensitive to the effects of whitespace on code clarity. Follow these guidelines related to whitespace:

- Use spaces instead of tabs for indentation.
- Use four spaces for each level of syntactically significant indenting.
- Lines should be 79 characters in length or less.
- Continuations of long expressions onto additional lines should be indented by four extra spaces from their normal indentation level.
- In a file, functions and classes should be separated by two blank lines.
- In a class, methods should be separated by one blank line.
- In a dictionary, put no whitespace between each key and colon, and put a single space before the corresponding value if it fits on the same line.
- Put one—and only one—space before and after the = operator in a variable assignment.
- For type annotations, ensure that there is no separation between the variable name and the colon, and use a space before the type information.

Type annotations werden sinds Python 3.5

Geintroduceerd om hints te geven welke data types men verwacht als argumenten en/of return types van functions of methods

```
# this piece of code demonstrates the Python type annotation feature
def greeting(name: str) -> str:
    return 'Hello ' + name

print(greeting(input('What is your name?\n')))
```

© 2008 Pearson Education, Inc. All rights reserved.

3.1 PEP-8 style guide

Naming conventions

- Functions, variables, and attributes should be in `lowercase_underscore` format.
- Protected instance attributes should be in `_leading_underscore` format.
- Private instance attributes should be in `__double_leading_underscore` format.
- Classes (including exceptions) should be in `CapitalizedWord` format.
- Module-level constants should be in `ALL_CAPS` format.
- Instance methods in classes should use `self`, which refers to the object, as the name of the first parameter.
- Class methods should use `cls`, which refers to the class, as the name of the first parameter.

Last slide on 02/03/222

© 2008 Pearson Education, Inc. All rights reserved.

3.1 PEP-8 style guide ctd.

Expressions and Statements

The Zen of Python states: "There should be one—and preferably only one—obvious way to do it." PEP 8 attempts to codify this style in its guidance for expressions and statements:

- Use inline negation (`if a is not b`) instead of negation of positive expressions (`if not a is b`).
- Don't check for empty containers or sequences (like `[]` or `''`) by comparing the length to zero (`if len(somelist) == 0`). Use `if not somelist` and assume that empty values will implicitly evaluate to `False`.
- The same thing goes for non-empty containers or sequences (like `[1]` or `'hi'`). The statement `if somelist` is implicitly `True` for non-empty values.
- Avoid single-line `if` statements, `for` and `while` loops, and `except` compound statements. Spread these over multiple lines for clarity.
- If you can't fit an expression on one line, surround it with parentheses and add line breaks and indentation to make it easier to read.
- Prefer surrounding multiline expressions with parentheses over using the \ line continuation character.

© 2008 Pearson Education, Inc. All rights reserved.

3.1 PEP-8 style guide ctd.

Imports

PEP 8 suggests some guidelines for how to import modules and use them in your code:

- Always put `import` statements (including `from x import y`) at the top of a file.
- Always use absolute names for modules when importing them, not names relative to the current module's own path. For example, to import the `foo` module from within the `bar` package, you should use `from bar import foo`, not just `import foo`.
- If you must do relative imports, use the explicit syntax `from . import foo`.
- Imports should be in sections in the following order: standard library modules, third-party modules, your own modules. Each subsection should have imports in alphabetical order.

© 2008 Pearson Education, Inc. All rights reserved.

3.1 Ins vs cls vs static methods

- **Standaard methods hebben toegang tot een specifieke instance van een klasse en kunnen hiervan desnoods attributen wijzigen omdat ze toegang hebben tot de instance via 'self'. Het zijn 'instance' methods. Ze kunnen enkel worden aangeroepen nadat een concreet object (class instance) is gecreerd**
- **Class methods hebben enkel toegang tot de klasse via 'cls' en kunnen geen attributen wijzigen. Ze worden gespecifieerd via de** @classmethod **decorator**
- **Static methods hebben noch toegang tot de instance, noch tot de klasse. Ze worden gespecifieerd via de** @staticmethod **decorator**

© 2008 Pearson Education, Inc. All rights reserved.

3.1 Ins vs cls vs static methods

```python
class Circle:
    """A simple 2d circle in the Eucledian plane"""
    def __init__(self, center=Point(), radius=0.):
        """ Initialisation of circle attributes. """
#        if radius < 0:
#            raise ValueError('radius of a circle must have positive
        self.radius = radius
        self.center = center
    @property
    def radius(self):
        """ Return the circle radius """
        return self._radius
    @radius.setter
    def radius(self, radius):
        """ Set the radius."""
        if radius < 0:
            raise ValueError('radius of a circle must have positive
        self._radius = radius
    @property
    def center(self):
        """ Return the circle center """
        return self._center
    @center.setter
    def center(self, center=Point()):
        """ Set the center coordinates."""
        self._center = center
    def surface(self):
        return (math.pi)*(self.radius)*(self.radius)
    def circumference(self):
        return 2.*(math.pi)*(self.radius)
```

```python
    def __str__(self):
        """ Converts a point into a string. """
        return "({0._center}, {0._radius})".format(self)
    @classmethod
    def unitcircle(cls):
        return cls(Point(),1.)
    @staticmethod
    def static_surf_circ(r):
        return (math.pi) * (r) * (r)
```

```python
# This is a test script that compares the use of
# instance methods
# class methods
# static methods
from Shapes import Circle, Point

# example calling a class method
print(Circle.unitcircle())
# example of calling a static method
radius=15.1234
print(f'the area of a circle with radius {radius:.2f} is: '
      f'{Circle.static_surf_circ(radius):.4f}')
# example of calling a static method
mypoint=Point(2.2, -1.5)
mycircle=Circle(mypoint, radius)
print(mycircle)
```

Voorbeeld: Shapes.py, TestShapes.py

3.1 Formatted strings

Python heeft verschillende manieren om strings te formatteren

Sinds Python 3.6 is de interpolated format string (f-string) ingevoerd: gebruik deze bij voorkeur (kortste en meest leesbaar)

```python
# This program illustrates working with formatted strings
# give preference to the new Python interpolated format strings
# Author: Nick van Remortel
# last revision: (dd/mm/yy=01/03/2021)
#

# First we make a list of tuples that contain a food item and a quatit
pantry = [
    ('avocados', 1.25),
    ('bananas', 2.5),
    ('cherries', 15),
]
for i, (item, count) in enumerate(pantry):
    # This is the old c-style string formatting
    old_style = '#%d: %-10s = %d' % (
        i + 1,
        item.title(),
        round(count),
    )
    # this uses the string.format() method
    new_style = '#{}: {:<10s} = {}'.format(
        i + 1,
        item.title(),
        round(count),
    )
    # this is the best and shortest way using python f-strings
    f_string = f'#{i+1}: {item.title():<10s} = {round(count)}'

    assert old_style == new_style == f_string
    print(f_string)
```

```
/home/uauser/code/mypython/bin/python /home/uauser/code/stringwork.py
#1: Avocados   = 1
#2: Bananas    = 2
#3: Cherries   = 15

Process finished with exit code 0
```

Verkies ook de enumerate() function boven de range() function om te itereren over lists die andere waarden bevatten dan simpele numerieke velden

Enumerate geeft als return waarden een iterator count en de waarde van de items waarover je itereert

Voorbeeld: stringwork.py

© 2008 Pearson Education, Inc. All rights reserved.

3.1 parallele iteraties met zip

Stel je hebt twee (aan elkaar gerelateerde) lijsten met een gelijk aantal elementen

Als je over beiden in parallel wil itereren gebruik je best de zip() functie

```python
# This program illustrates the use of the zip() function
# to iterate in parallel over two  lists of the same length

# First we create a list of 3 names
names = ['Cecilia', 'Lise', 'Marie']
# let's count the number of characters in each name of this list
# this can be done in various ways
# first way: Loops
cts = []
for i in range(len(names)):
    cts.append(len(names[i]))

print(cts)
# second way, with maps
def countchar(name):
    return len(name)
ctts = map(countchar, names)
print(list(ctts))
# Third way (and best way):
# list comprehensions
counts = [len(n) for n in names]
print(counts)
```

```python
longest_name = None
max_count = 0
for i in range(len(names)):
    count = counts[i]
    if count > max_count:
        longest_name = names[i]
        max_count = count

print(longest_name)

# second way, using enumerate
longest_name = None
max_count = 0
for i, name in enumerate(names):
    count = counts[i]
    if count > max_count:
        longest_name = name
        max_count = count

print(longest_name)

# third way, using zip
longest_name = None
max_count = 0
for name, count in zip(names, counts):
    if count > max_count:
        longest_name = name
        max_count = count

print(longest_name)
```

Voorbeeld: zipping.py

© 2008 Pearson Education, Inc. All rights reserved.

3.1 List comprehensions

In voorgaand voorbeeld is het gebruik van een list comprehension getoond

Structuur: new_list = [expression for member in iterable]

met:

- expression: **Kan de member zelf zijn, een method call, of eender welke expression die een value teruggeeft**
- member: **het object of de value in de iterable data structuur**
- iterable: **een itereerbare data structuur: list, set, sequence, generator, of eender wel object dat zijn elementen een voor een kan teruggeven**

Meer weten: https://realpython.com/list-comprehension-python/

© 2008 Pearson Education, Inc. All rights reserved.

3.1 Catch all unpacking

De algemene regel is om indexing te vermijden en waar mogelijk elementen uit een lijst te gebruiken via multiple assignment unpacking

In sommige gevallen ben je enkel geïnteresseerd in een klein aantal elementen uit een lijst, maar moet je toch de rest kwijt in een nieuw lijstje: gebruik catch-all unpacking via de *operator

```python
# This code example demonstrates catch-all unpacking
# Author : Nick van Remortel
# revised on 02/03/2022

# First make a list
car_ages = [0, 9, 4, 8, 7, 20, 19, 1, 6, 15]
# now sort it from high to low
car_ages_descending = sorted(car_ages, reverse=True)
# this ststement below will result in an error
# oldest, second_oldest = car_ages_descending
# let's do it this way then (not optimal)
oldest = car_ages_descending[0]
second_oldest = car_ages_descending[1]
others =car_ages_descending[2:]
print(oldest, second_oldest, others)
# now use catch-all unpacking
oldest, second_oldest, *others = car_ages_descending
print(oldest, second_oldest, others)
# we can use starred expressions in any position
oldest, *others, youngest = car_ages_descending
print(oldest, youngest, others)
```

```
/home/uauser/code/mypython/bin/python /home/uauser/code/unpacking.py
20 19 [15, 9, 8, 7, 6, 4, 1, 0]
20 19 [15, 9, 8, 7, 6, 4, 1, 0]
20 0 [19, 15, 9, 8, 7, 6, 4, 1]

Process finished with exit code 0
```

Voorbeeld: unpacking.py

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

- **Functie namen zijn references naar functies die in het geheugen zijn opgeslagen**
- **Je kan meerdere namen toewijzen aan dezelfde functie**
- **Functies kunnen ook als parameter doorgegeven worden aan andere functies**
- **Functies kunnen ook functies teruggeven als return parameter**

```
import math
# a function can have multiple names
def succ(x):
    return x + 1
successor = succ
print(successor(10))
# we can even delete the original name
# and keep using the new one
del succ
print(successor(10))
# functions passed as arguments

def foo(func):
    print("The function " + func.__name__ + " was passed to foo")
    res = 0
    for x in [1, 2, 2.5]:
        res += func(x)
    return res
print(foo(math.sin))
print(foo(math.cos))
```

Voorbeeld: functions.py

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

- **Functies kunnen binnen andere functies gedefinieerd worden (lokale helper functies)**

  Let op de scope van variabelen binnen deze geneste functies:

  Python functies ondersteunen closures: functies kunnen naar variabelen refereren die uit de scope komen van waar de functie gedefinieerd is. Maar let op: indien je een nieuwe assigment maakt van een variabele uit die scope, maak je een lokale versie die de vorige versie overschaduwt.

```
def sort_priority(numbers, group):
    found = False
    def helper(x):
        # nonlocal found # Investigate the effect of this statement
        if x in group:
            found = True # this variable assignment enforces local scope
            return (0,x)
        return (1, x)
    numbers.sort(key=helper)
    return found
```

group is gekend omwille van het closure principe

found staat in een assignment waardoor het een lokale scope krijgt

Deze return zal altijd de found waarde uit de scope van de buitenste functie geven, tenzij je hem nonlocal maakt

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

- **Functies kunnen meerdere waarden teruggeven: Dit gebeurt standaard via een tuple**

Vermijd echter het teruggeven van meer dan 3 waarden die je dan via unpacking gaat toewijzen aan andere variabelen in je programma

Je kan ook weer catch-all unpacking gebruiken

Indien je toch veel resultaten wil terug geven: schrijf een klasse!

- **Er bestaat ook de mogelijkheid om een arbitrair aantal argumenten van een functie als optioneel te beschouwen, dut gebeurt via star argumenten: *args**

```python
def get_stats(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    return minimum, maximum
# A function that returns a list with unknown length
def get_avg_ratio(numbers):
    average = sum(numbers) / len(numbers)
    scaled = [x / average for x in numbers]
    scaled.sort(reverse=True)
    return scaled
# A function with variable positional arguments
def log(message, *values):
    if not values:
        print(message)
    else:
        values_str = ', '.join(str(x) for x in values)
        print(f'{message}: {values_str}')
```

```python
# First define a list of numbers
lengths = [63, 73, 72, 60, 67, 66, 71, 61, 72, 70]

# unpacking the output tuple given by get_stats
minimum, maximum = get_stats(lengths)
print(f'Min:  {minimum}, Max: {maximum}')

# unpacking the output of get_avg_ratio by
# catch-all unpacking
longest, *middle, shortest = get_avg_ratio(lengths)
print(f'Longest:  {longest:4.0%}')
print(f'Shortest:  {shortest:4.0%}')

# using variable position argument function log()
log('My lengths are', lengths)
log('No lengths')
```

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

- **Functie argumenten worden standaard herkend via hun positie in de argumentenlijst (positional arguments). Je kan ook een variabel aantal positional arguments meegeven (zie vorige slides)**
- **Je kan ook functie argumenten meegeven volgens de naam die ze hebben in de functie definitie (keyword arguments). Dit kan ambiguïteiten in de volgorde van de argumenten vermijden.**
- **Je hebt onder normale omstandigheden bij het aanroepen van een functie als gebruiker de keuze om ofwel positional of keyword, argumenten, of een mix van beiden te gebruiken**
- **Je kan ook een dictionary gebruiken om keyword argumenten door te geven via de ** operator bij het aanroepen van de functie**
- **Je kan ook een functie definieren met een arbitrair aantal keyword argumenten door de ** operator te gebruiken in de argumentenlijst van de functie definitie**
- **Optionele argumenten kunnen een default waarde krijgen in de functie definitie**
- **Het is aan te bevelen om optionele argumenten altijd via hun keyword door te geven**
- **Indien je bij het definieren van een functie wil afdwingen dat er enkel keyword of enkel positional arguments worden meegegeven, gebruik dan * en/of / in de argumentenlijst**

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

```python
def remainder(number, divisor):
    return number % divisor

# calling the function with positional arguments
print(f'computing the remainder of 20/7: {remainder(20,7)}')
# calling the function with keyword arguments
print(f'computing the remainder of 20/7: {remainder(number=20, divisor=7)}')
# when using keyword arguments, you can change the position and still
# get the right result
print(f'computing the remainder of 20/7: {remainder(divisor=7, number=20)}')
# you can also mix positional and keyword arguments
# but positional arguments must always come first
print(f'computing the remainder of 20/7: {remainder(20, divisor=7)}')
my_kwargs = {
    'number': 20,
    'divisor': 7,
}
# now pass the dictionary to the function as keyword arguments
# with the ** operator
print(f'computing the remainder of 20/7: {remainder(**my_kwargs)}')
def print_parameters(**kwargs):
    for key, value in kwargs.items():
        print(f'{key} = {value}')

# the three parameters below will be collected
# into a dictionary kwargs via the catch-all
# parameter mechanism and the ** operator
print_parameters(alpha=1.5, beta=9, gamma=4)
```

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Functions

- **Een gecombineerd voorbeeld**

```
# Combining some aspects together
# a function that generates polynomials
def polynomial_creator(*coeffs):
    """ coefficients are in the form a_n, a_n_1, ... a_1, a_0
    """
    def polynomial(x):
        res = coeffs[0]
        for i in range(1, len(coeffs)):
            res = res * x + coeffs[i]
        return res
    return polynomial
p1 = polynomial_creator(4)
p2 = polynomial_creator(2, 4)
p3 = polynomial_creator(1, 8, -1, 3, 2)
p4 = polynomial_creator(-1, 2, 1)
print('Calculating polynomials')
for x in range(-2, 2, 1):
    print(x, p1(x), p2(x), p3(x), p4(x))
```

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Function decorators

- **Een decorator is een callable python object (je kan het aanroepen als een functie) dat gebruikt wordt om het gedrag van een functie, method of class definition aan te passen**
- **Een reference naar een functie func of class C wordt doorgegeven aan de decorator en de decorator kan dan een gemodificeerde versie van de functie teruggeven**

```python
def argument_test_natural_number(f):
    def helper(x):
        if type(x) == int and x > 0:
            return f(x)
        else:
            raise Exception("Argument is not an integer")
    return helper
@argument_test_natural_number
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)

for i in range(1,10):
    print(i, factorial(i))
```

```python
# you can even use multiple decorators
# the outside decorator is applied first
def call_counter(func):
    def helper(*args, **kwargs):
        helper.calls += 1
        return func(*args, **kwargs)
    helper.calls = 0
    return helper

@call_counter
@argument_test_natural_number
def factorial2(n):
    if n == 1:
        return 1
    else:
        return n * factorial2(n-1)

for i in range(1,10):
    print(i, factorial2(i), f'called {factorial2.calls:d} times')
```

© 2008 Pearson Education, Inc. All rights reserved.

3.3 List Comprehensions vs Generators

- Veel programma's bevatten lists, dictionaries en sets
- Comprehensions: specifieke syntax om door deze data types te itereren en nieuwe afgeleide data structuren te maken
- Syntax: newlist = [*expression* for *item* in *iterable* if *condition* == True]
  - Item: een element uit de itereerbare collectie
  - Expression: het huidige item uit de iterable, of het resultaat van een functie uitgevoerd op dit item, en/of een combinatie van selectie statements
  - Iterable: Een itereerbaar object zoals list, tuple, dict, set, of het resultaat van een functie die een itereerbaar object teruggeeft (bvb. range())
  - Condition: een logische voorwaarde die moet voldaan zijn, die werkt als filter op de items en die enkel items accepteert als de conditie waar is

• Comprehensions ondersteunen meerder if condities, maar je beperkt je best to maximum twee

• Wanneer je verwacht dat er grote lists gaan worden gegenereerd, kan je overwegen een generator te gebruiken

• Generators worden gemaakt met functies die een yield expression bevatten: expressions die een itereerbare serie van output waarden teruggeven

• Je kan ook *generator expressions* maken door list-comprehension syntax tussen ronde haakjes () te zetten

• Check ook de *itertools* built-in module

© 2008 Pearson Education, Inc. All rights reserved.

3.3 List Comprehensions

```python
# First make a list
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []
# This is the old fashioned way of iterating
# we search for words containing the letter a
for x in fruits:
  if "a" in x:
    newlist.append(x)

print(newlist)

# An alternative is using lambda expressions
newlist2 = list(filter(lambda x: "a" in x, fruits))
print(newlist2)
# You can so the same thing much shorter
# via comprehensions
newlist3 = [x for x in fruits if "a" in x]
print(newlist3)
```

```python
# also dictionaries can work with comprehensions
dict1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
# Double each value in the dictionary
double_dict1 = {k:v*2 for (k,v) in dict1.items()}
print(double_dict1)
# Double each key in the dictionary
dict1_keys = {k*2:v for (k,v) in dict1.items()}
print(dict1_keys)
```

```
/home/uauser/code/mypython/bin/python /home/uauser/code/comprehensions.py
['apple', 'banana', 'mango']
['apple', 'banana', 'mango']
['apple', 'banana', 'mango']
{'a': 2, 'b': 4, 'c': 6, 'd': 8, 'e': 10}
{'aa': 1, 'bb': 2, 'cc': 3, 'dd': 4, 'ee': 5}
```

Voorbeeld: comprehensions.py

© 2008 Pearson Education, Inc. All rights reserved.

3.3 Generators

```python
# Looping through a piece of text and finding the
# index of each new word
# keep in mind that this can be a lot of text

def index_words(text):
  result = []
  if text:
      result.append(0)
  for index, letter in enumerate(text):
      if letter == ' ':
        result.append(index + 1)
  return result
address = 'The quick brown fox jumps over the lazy dog'
result = index_words(address)
print(result[0:10])

# now we make a generator function
def index_words_iter(text):
  if text:
    yield 0
  for index, letter in enumerate(text):
    if letter == ' ':
      yield index + 1

it = index_words_iter(address)
print(next(it))
print(next(it))
```

```python
# another example of a generator
# A simple generator for Fibonacci Numbers
def fib(limit):
  # Initialize first two Fibonacci Numbers
  a, b = 0, 1

  # One by one yield next Fibonacci Number
  while a < limit:
    yield a
    a, b = b, a + b


# Create a generator object
x = fib(5)
print('Fibonacci\n')
print(next(x))
print(next(x))
print(next(x))
print(next(x))

# Illustration of the use of generator expressions
it = (len(x) for x in open('ocean-heat.csv'))
print(next(it))
print(next(it))
# This can be combined with yet another
# generator expression
# keep in mind that the previous
# iterator keeps advancing as well
# like a domino effect
roots = ((x, x**0.5) for x in it)
print(next(roots))
print(next(roots))
```


erved.

3.4 Exception handling

- **Er bestaan twee soorten fouten in programma's: Syntax Errors en Exceptions**
- **Syntax errors worden door de compiler/interpreter herkend bij het parsen van de broncode**
- **Exceptions onstaan bij syntactisch correcte code, maar bij het onstaan van fouten bij het uitvoeren van de code zelf**
- **Exceptions zorgen voor een verandering in de flow van het programma: je kan zelf ingrijpen in deze flow naargelang het sort van exception**
- **Python kent een heleboel verschillende soorten exceptions: voor info bekijk https://docs.python.org/3/library/exceptions.html**
- **Alle exceptions zijn instances van klassen die uiteindelijk allemaal afgeleid zijn van de `BaseException` class**
- **Je an zelf ook eigen exceptions definieren (door te erven van de bestaande python exception classes)**

© 2008 Pearson Education, Inc. All rights reserved.

3.4 De try, except, else clauses

- **Try statements worden geplaatst wanneer men vermoed dat een code block exceptions kunnen genereren (bvb. Bestanden kunnen niet geopend, rekenkundige fouten, onmogelijke conversies, …)**
- **Elke try clause wordt gevolgd door een of meerdere except clauses, ook exception handlers genoemd, die het type exception specifieren waarvoor bepaalde acties moeten worden ondernomen**
- **De else clause specifieert code die wordt uitgevoerd wanneer er geen exceptions worden gegenereerd in de try suite**
- **Wanneer een exception voorkomt in een try block wordt op het punt waar de exception wordt gegenereerd (het raise point), de verdere uitvoering van de try suite gestopt en controle overgedragen aan de overeenkomstige except suite**
- **Indien er geen exception handlers gevonden worden die overeen komen met de exception gegenereerd in de try suite dan worden er default handlers aangeroepen**
- **Je kan meerdere exceptions behandelen in de zelfde except clause**
- **Elke try, except … else blok kan beeindigd worden met een finally clause: de suite van de finally clause wordt gegarandeerd als laatste uitgevoerd, ongeacht of er een exception werd gegenereerd of niet**
- **Je kan ook je eigen exceptions genereren via het raise statement**

© 2008 Pearson Education, Inc. All rights reserved.

3.4 Exception handling

```python
# First make a function that divides two numbers
def divider(numerator, denominator):
    return int(numerator)/int(denominator)

# Execute this for the problematic cause
# division by zero

#print(divider(5, 0))
# try to do other decent things
# will this execute?
print(divider(5, 10))

# now put the statemnt in a try block

try:
    print(divider(5, 0))
except:
    print('something went wrong')
else:
    print('all going well')

print(divider(5, 10))
```

```python
# you can be more specific and specify the
# possible exceptions

try:
    print(divider(5, 0))
except ZeroDivisionError:
    print('We know you are trying to divide by zero!')
else:
    print('all going well')

print(divider(5, 10))

# let's include some other weird possibilities
# now we also include a finally cluase
try:
    print(divider('hello', 5))
except ZeroDivisionError:
    print('We know you are trying to divide by zero!')
except ValueError:
    print('You are not using the right type')
else:
    print('all going well')
finally:
    print('this statement will always execute')

print(divider(5, 10))
```

Voorbeeld: exceptions.py

© 2008 Pearson Education, Inc. All rights reserved.

3.4 Exception handling

```python
# below we will raise our own exception
def yourname(name):
    if name != 'Nick':
        raise NameError('Not the right name')
    else:
        return name

try:
    print(yourname('Jack'))
except NameError:
    print('We know you are using the wrong name!')
else:
    print('all going well')
finally:
    print('this statement will always execute')
```

```
0.5
something went wrong
0.5
We know you are trying to divide by zero!
0.5
You are not using the right type
this statement will always execute
0.5
We know you are using the wrong name!
this statement will always execute

Process finished with exit code 0
```

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Polymorfisme

- **Komt van het woord ‘poly’ =meerdere, ‘morph’=vorm: het voorkomen van gelijkaardig gedrag bij verschillende vormen**
- **In Object Georienteerd programmeren (OOP) refereert polymorfisme naar een entiteit van een bepaald type (functie, method, operator of object) dat verschillende types representeert in verschillende scenarios**
- **Omdat python geen compile-time type checking doet leent de taal zich uitermate op een flexibele manier tot polyformisme**
- **Op niveau van klassen wordt polymorfisme het beste afgedwongen via inheritance van een abstract base class (ABC) die de interfaces vastlegt (de namen en argumenten van de publieke methods), maar geen implementaties voorziet. Implementaties gebeuren dan in subclasses die erven van dezelfde ABC**

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Polymorfisme

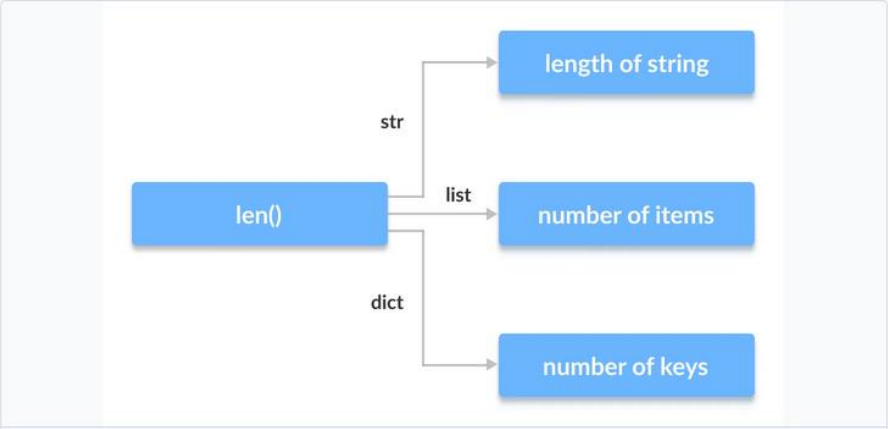
- **Vele operatoren en built-in functions vertonen verschillend gedrag dat afhangt van het type en aantal inputs**

```
num1 = 1
num2 = 2
print(num1+num2)

str1 = "Hello "
str2 = "World!"
print(str1+str2)

# Now we show the polymorphic behavior of the
# len() function

print(len("program"))
print(len(["Pyhon", "Java", "C"]))
print(len({"name": "john", "address": "Nepal"}))
```



Voorbeeld: polymorphism.py

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Class Polymorfisme

- **Indien je in verschillende classes dezelfde interface (via zelfde method name) voorziet, krijg je polymorfisme op class niveau**

```python
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"I am a cat. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Meow")


class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"I am a dog. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Bark")
```

```python
cat1 = Cat("Kitty", 2.5)
dog1 = Dog("Fluffy", 4)

for animal in (cat1, dog1):
    animal.make_sound()
    animal.info()
    animal.make_sound()
```

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Classes

- **We brengen nu het voorgaande bij elkaar: type annotators, custom decorators, built_in decorators, polymorfisme, …**

```python
from __future__ import annotations
import math
from collections import namedtuple

# Define a decorator that can wrap
# Any class method
def argument_test_natural_number(f):
    def helper(self, x):
        if (type(x) == int or type(x) == float) and x > 0:
            return f(self, x)
        else:
            raise Exception("Argument is not an integer or fl

    return helper
```

```python
class Point(object):
    """An (x,y) coordinate in 2D Euclidian space"""

    def __init__(self, cx: float =0., cy: float =0.):
        """ Initialisation of Euclidian point attributes.
        :type cx: float
        :type cy: float
        """
        self.x = cx
        self.y = cy

    @property
    def x(self):
        """ Return the x coordinate """
        return self._x

    @x.setter
    def x(self, cx: float):
        """ Set the x coordinate."""
        self._x = cx
```

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Classes

```python
    @property
    def y(self) -> float:
        """ Return the x coordinate """
        return self._y

    @y.setter
    def y(self, cy: float):
        """ Set the y coordinate."""
        self._y = cy

    def __str__(self) -> str:
        """ Converts a point into a string. """
        return f'({self_x}, {self._y})'

    def __repr__(self) -> str:
        """ Represents a point as a string. """
        return f'({self_x}, {self._y})'

    def __add__(self, other) -> Point:
        """ Adds two points """
        return Point(self._x + other._x, self._y + other._y)

    def __sub__(self, other) -> Point:
        """ Adds two points """
        return Point(self._x - other._x, self._y - other._y)
```

```python
class Circle(object):
    """A simple 2d circle in the Eucledian plane"""

    def __init__(self, center: Point =Point(), radius: float =0.):
        """ Initialisation of circle attributes. """

        self.radius = radius
        self.center = center

    def __str__(self):
        """ Converts a point into a string. """
        return f'({self._center}, {self._radius})'

    @property
    def radius(self) -> float:
        """ Return the circle radius """
        return self._radius


    @radius.setter
    @argument_test_natural_number
    def radius(self, radius):

        self._radius = radius

    @property
    def center(self) -> Point:
        """ Return the circle center """
        return self._center
```

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Classes

```python
    @center.setter
    def center(self, center: Point =Point()):
        """ Set the center coordinates."""
        self._center = center

    def surface(self) -> float:
        """ return the surface area"""
        return (math.pi)*(self._radius)*(self._radius)

    def circumference(self) -> float:
        """return the circumference"""
        return 2.*(math.pi)*(self._radius)

    @classmethod
    def unitcircle(cls) -> Circle:
        return cls(Point(),1.)

    @staticmethod
    def static_surf_circ(r: float =0.) -> float:
        return (math.pi) * (r) * (r)
```

```python
class Zircle(Point):
    """A simple 2d circle that inherits from Point"""


    def __init__(self, c1: float=0., c2: float=0., radius: float=0.):
        """ Initialisation of circle attributes. """
        super().__init__(c1,c2)
        self.radius = radius  #validate via property

    def __str__(self) ->str:
        """ Converts a point into a string. """
        return f'Circle ({super().__str__()}, {self._radius})'

    @property
    def radius(self) ->float:
        """ Return the circle radius """
        return self._radius

    @radius.setter
    @argument_test_natural_number
    def radius(self, radius: float):
        """ Set the radius."""
        self._radius = radius
    @property
    def surface(self) -> float:
        """ returns the surface area of the circle """
        return (math.pi)*(self.radius)*(self.radius)
```

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Classes

```python
class Square(Point):
    """A simple 2d circle that inherits from Point"""

    def __init__(self, c1: float=0., c2: float=0., height: float=0.):
        """ Initialisation of circle attributes. """
        super().__init__(c1,c2)
        self.height = height  #validate via property

    def __str__(self) ->str:
        """ Converts a point into a string. """
        return f'Square ({super().__str__()}, {self._height})'

    @property
    def height(self) ->float:
        """ Return the circle radius """
        return self._height

    @height.setter
    @argument_test_natural_number
    def height(self, height: float=0.):
        """ Set the radius."""
        self._height = height
    @property
    def surface(self) -> float:
        """ returns the surface area of the circle """
        return self._height*self._height
    @property
    def circumference(self) -> float:
        """ returns the circumference of teh circle """
        return 4.*self._height
```

```python
from Shapes import Circle
from Shapes import Zircle
from Shapes import Square
from Shapes import Point

# example calling a class method
print(Circle.unitcircle())
# example of calling a static method
radius=2.
print(f'the area of a circle with radius {radius:.2f} is: '
      f'{Circle.static_surf_circ(radius):.4f}')
# example of calling a static method
mypoint=Point(2.2, -1.5)
mycircle=Circle(mypoint, radius)
print(mycircle)

# Testing polymorphic class behavior
p1=Point(0.5, 0.7)
p2=Point(-0.2, 2.)
mycirc=Zircle(-0.2,3.0,2.0)
mysquare=Square(0.8, 2.7, 2.0)
print(mycirc)
for shape in (mycirc, mysquare):
      print(f'I made a {shape}, with surface '
            f'{shape.surface:10.4f} and circumference '
            f'{shape.circumference:10.4f}')
```

© 2008 Pearson Education, Inc. All rights reserved.

3.6 Performance

- **Het dynamische karakter van Python kan verrassend gedrag vertonen in termen van performantie: Operaties die langzaam of complex lijken, worden door gebruik van juiste programeerstijl net heel efficient uitgevoerd (bvb. comprehensions en generators)**
- **Het is, zeker voor onervaren programmeurs, vrij lastig om de computationele bottlenecks van een python programma te identificeren**
- **Je kan de performantie van code meten met een *profiler***
- **Python voorziet zelf in twee profilers: profile en cProfile (cProfile is beter omdat het zelf de minste overhead produceert)**

© 2008 Pearson Education, Inc. All rights reserved.

3.6 cProfile

```python
from random import randint
from cProfile import Profile
from pstats import Stats

def insertion_sort(data):
    result = []
    for value in data:
        insert_value(result, value)
    return result

def insert_value(array, value):
    for i, existing in enumerate(array):
        if existing > value:
            array.insert(i, value)
            return
        array.append(value)

max_size = 10**4
data = [randint(0, max_size) for _ in range(max_size)]
test = lambda: insertion_sort(data)

# Set up the profiler
profiler = Profile()
profiler.runcall(test)

stats = Stats(profiler)
stats.strip_dirs()
stats.sort_stats('cumulative')
stats.print_stats()
```

```
         10003 function calls in 0.046 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.046    0.046 profiling.py:23(<lambda>)
        1    0.015    0.015    0.046    0.046 profiling.py:8(insertion_sort)
    10000    0.031    0.000    0.031    0.000 profiling.py:14(insert_value)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

ncalls: aantal calls naar betreffende functie tijdens profiling

tottime: aantal seconden nodig om functie uit te voeren, zonder de tijd nodig om andere functies uit te voeren die deze functie aanroept

percall: gemiddeld aanta;l seconden om de functie uit te voeren

cumtime: cumulatieve tijd nodig om de functie uit te voeren, inclusief het aanroepen van andere functies erbinnen

percall: gemiddeld aantal seconden van de cumulatieve tijd nodig …

Voorbeeld: profiling.py

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Testing & Debugging

- **Python heeft geen compile-time static type checking (zoals C, C++, …), hetgeen voor en nadelen heeft: het is compact en efficient om te programmeren, maar …**
- **Je weet nooit 100% zeker of de functies die door je programma worden aangeroepen goed gedefinieerd zijn op runtime niveau**
- **Oplossing: testen, testen, testen!**
- **Algemene Tips:**
  - **Implementeer de __repr__ dunder method in je klassen en gebruik de repr() functie om de fundamentele representatie van een object te weten**
  - **Gebruik type annotations waar het nuttig is**
  - **Schrijf docstrings voor elke functie, class en module**
  - **Gebruik een virtual environment**
  - **Gebruik de unittest built-in module**
  - **Gebruik een debugger (pdb of Pycharm debugger: https://www.jetbrains.com/help/pycharm/debugging-code.html#general-procedure)**

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Unittest

```
uauser@VMFys:~$ ls
code      Documents  file4.txt  Pictures  PycharmProjects  snap       tmp
Desktop   Downloads  Music      Public    scripts          Templates  Videos
uauser@VMFys:~$ cd code/
uauser@VMFys:~/code$ ls
comprehensions.py  ocean-heat.csv  pyscript1      TestShapes.py    zipping.py
exceptions.py      profiling.py    pyscript1~     test_tokens.py
functions.png      prog1.py        Shapes.py      test_tokens.py~
functions.py       prog1.py~       simpleplot.py  type_annot.py
mypython           __pycache__     stringwork.py  unpacking.py
uauser@VMFys:~/code$ cd mypython/
uauser@VMFys:~/code/mypython$ source ./bin/activate
(mypython) uauser@VMFys:~/code/mypython$ python
Python 3.9.5 (default, Nov 23 2021, 15:27:38)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> assert sum([1, 2, 3]) == 6
>>> assert sum([1, 2, 3]) == 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
>>>
```

assert statement wordt gebruikt in testen en debuggen: het gaat na of een logische conditie vervuld is en genereert een exception AssertionError, wanneer de conditie False is

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Unittest

Stop je tests in simpele test scripts

```python
# Sum() is a built-in function that works with
# any iterable structure
# Let's test it with a list and a tuple

def test_sum():
    assert sum([1, 2, 3]) == 6, "Should be 6"

def test_sum_tuple():
    assert sum((1, 2, 2)) == 6, "Should be 6"

if __name__ == "__main__":
    test_sum()
    test_sum_tuple()
    print("Everything passed")
```

```
/home/uauser/code/mypython/bin/python /home/uauser/code/test_sum.py
Traceback (most recent call last):
  File "/home/uauser/code/test_sum.py", line 18, in <module>
    test_sum_tuple()
  File "/home/uauser/code/test_sum.py", line 14, in test_sum_tuple
    assert sum((1, 2, 2)) == 6, "Should be 6"
AssertionError: Should be 6

Process finished with exit code 1
```

Voorbeeld: test_sum.py

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Unittest

- Unittest is een test framework uit de python standard library
- Unittest eist dat je je tests in een class met specifieke methods stopt: erft van unittest.TestCase class en assertion methods ipv. De eerder gebruikte assert() functie
- Alle methods in je test class die beginnen met test_ worden automatisch uitgevoerd en op een gestructureerde manier gerapporteerd

```python
import unittest


class TestSum(unittest.TestCase):

    def test_sum(self):
        self.assertEqual(sum([1, 2, 3]), 6, "Should be 6")

    def test_sum_tuple(self):
        self.assertEqual(sum((1, 2, 2)), 6, "Should be 6")

if __name__ == '__main__':
    unittest.main()
```

```
Ran 2 tests in 0.011s

FAILED (failures=1)
Launching unittests with arguments python -m unittest /home/uauser/code/test_sum_unittest.py in /home/uau


Should be 6
6 != 5

Expected :5
Actual   :6
<Click to see difference>

Traceback (most recent call last):
  File "/home/uauser/code/test_sum_unittest.py", line 15, in test_sum_tuple
    self.assertEqual(sum((1, 2, 2)), 6, "Should be 6")
AssertionError: 5 != 6 : Should be 6



Process finished with exit code 1
```

Voorbeeld: test_sum_unittest.py

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Unittest assertions

- **Standaard assertion methods**

Method	Equivalent to
`.assertEqual(a, b)`	`a == b`
`.assertTrue(x)`	`bool(x) is True`
`.assertFalse(x)`	`bool(x) is False`
`.assertIs(a, b)`	`a is b`
`.assertIsNone(x)`	`x is None`
`.assertIn(a, b)`	`a in b`
`.assertIsInstance(a, b)`	`isinstance(a, b)`

`.assertIs()`, `.assertIsNone()`, `.assertIn()`, and `.assertIsInstance()` all have opposite methods, named `.assertIsNot()`, and so forth.

- **Indien je wil checken of de juiste exception wordt gegenereerd, test met assertRaises([exception])**

© 2008 Pearson Education, Inc. All rights reserved.

3.7 Unittest

- **Indien je voor het uitvoeren van test methods een aantal zaken wil klaarmaken (initilisatie van een object, openen van een file, database, etc.) kan je de `setUp` method implementeren**
- **Analoog kan je ervoor zorgen dat op het einde een cleanup gebeurt (objecten vernietigen, files of databases afsluiten, ..) kan je de `tearDown` method implementeren**
- **Je kan via class method decorators aageven wanneer bepaalde tests moeten worden overgeslagen (via skip, skipIf, skipUnless, …)**
- **Meer info zie: https://docs.python.org/3/library/unittest.html**
- **https://realpython.com/python-testing/**

© 2008 Pearson Education, Inc. All rights reserved.

Te Onthouden

1. Pythonic programming style
2. Geavanceerde concepten: o.m. comprehensions
3. Principes van Exception handling
4. Polymorfisme
5. Profiling en debugging
6. Unit testing

© 2008 Pearson Education, Inc. All rights reserved.